

Team ID: 6ab381dcd1db34949b43874a
Team Lead: Appase sanam sajjath Hussein
Members: Tamilselvi.M | Priyadarshini | Yuvasree | Yuvashini

Pocket Smart AI: Your Smart Budget and Recommendation Assistant

Team ID: 6ab381dcd1db34949b43874a

Team Lead: Appasi sanam sajjath hussein.s

Members:

- Tamilselvi.M
- Priyadarshini
- Yuvasree
- Yuvashini

DEFINE PROBLEM STATEMENTS

Maximum Marks: 3 Marks

Project Initialization and Planning Phase

Define Problem Statements (Customer Problem Statement Template)

Create a problem statement to understand your customer's point of view. The template helps focus on what matters when creating useful experiences for customers.

About the Project:

Pocket Smart AI is a smart budget and recommendations assistant designed to help users organize their income and expenses, understand spending patterns, manage a budget, and receive useful recommendations based on their financial information.

Customer Problem Statements:

Problem Statement	I am (Customer)	I'm trying to	But	Because	Which makes me feel
PS-1	A student or young user who wants to manage money better.	Track my daily spending and stay within a budget.	I do not always know where my money is going.	Expenses can be spread across different purchases and are difficult to monitor manually.	Confused and concerned about overspending.
PS-2	A user with a limited or fixed monthly income.	Create a practical budget and control unnecessary spending.	It is difficult to decide how much I should allocate to different needs.	Without a clear budget and spending summary, financial decisions become harder.	Stressed and uncertain about managing money.
PS-3	A user who wants personalized financial guidance.	Understand my spending and get useful recommendations.	Generic advice may not match my actual spending pattern.	Recommendations are more useful when they consider my own budget and expense information.	Hopeful but unsure about what action to take.

CUSTOMER JOURNEY MAP

Maximum Marks: 2 Marks

Map out the customer's experience stage-by-stage, capturing actions, touchpoints, thoughts and feelings, along with process ownership and improvement opportunities.

Phase of Journey	Stage 1	Stage 2	Stage 3
Actions	Find Pocket Smart AI; create an account; enter financial details.	Enter income, expenses and budget; view spending information.	Review AI recommendations; monitor budget; adjust spending habits.
Touchpoint	Website/App; sign-up page; financial input form.	Dashboard; budget/expense section; spending summary.	Recommendation section; reports/progress; updated budget information.
Customer Thought	I want a simple budgeting solution. Is my information safe?	I want to understand my spending. Is my budget realistic?	What should I change? Can I save more or reduce unnecessary spending?
Customer Feeling	Curious and hopeful	Engaged and concerned about spending	More informed and motivated
Process Ownership	Product / UI Team	Backend / Development Team	Support / Development Team
Opportunities	Make sign-up quick; explain privacy clearly; use simple input fields.	Make expense entry easy; provide clear summaries; show budget status.	Provide useful recommendations; make progress easy to understand; support better budgeting decisions.

CODE-LAYOUT, READABILITY AND REUSABILITY

S.No	Code Quality Parameter	Description	Followed	Remarks
1	Consistent Indentation	Uniform spacing/tabs used throughout the code	Yes	Consistent indentation improves readability across Python, HTML, CSS and JavaScript files.
2	Proper File Structure	Files and folders are logically organized	Yes	The project separates backend/application logic, static files, templates and configuration/dependency files.
3	Meaningful Variable Names	Variables reflect their purpose clearly	Yes	Names should relate clearly to users, income, expenses, budgets, recommendations and application functions.
4	Function / Method Names	Functions are descriptively named	Yes	Functions use names that indicate their purpose and make the code easier to understand.
5	Code Comments	Inline and block comments explain logic	Partial	Comments are useful where needed; complex financial/recommendation logic can be documented further.
6	Modular Design	Code is split into reusable functions/modules	Yes	FastAPI routes and application logic are organized into reusable parts.
7	No Redundant Code	Duplicate or unused code is removed	Yes	The main application flow is organized to avoid unnecessary duplication.
8	Error Handling	Exceptions and errors are handled gracefully	Yes	Invalid requests and API/application failures should be handled gracefully.

Reusable Components / Modules

S.No	Component / Module Name	Language / Technology	Where Reused	Reusability Level
1	FastAPI application/routes	Python / FastAPI	User authentication, budget, expense and recommendation requests	High
2	HTML templates	HTML / Jinja2	Login, registration, budget/expense and result pages	High
3	Static styling and scripts	CSS / JavaScript	Common UI styling and client-side interactions	High
4	Database models and access	Python / SQLAlchemy + SQLite	User, budget, expense and recommendation data operations	High
5	Gemini integration module	Python / Google GenAI SDK	AI recommendation generation	High

Overall Code Quality Assessment

Aspect	Rating (1-5)	Comments
Code Layout & Structure	3	Clear separation of backend, frontend, static files, templates and supporting components.
Readability	4	Meaningful naming and organization make the main application flow easy to follow.
Reusability	4	Common application, UI, database and AI integration components can be reused or extended.
Documentation / Comments	3	Basic documentation/comments are present; detailed comments can be added for complex sections.

Team ID: 6ab381dcd1db34949b43874a
Team Lead: Appase sanam sajjath Hussein

Members: Tamilselvi.M Priyadarshini Yuvasree Yuvashini		
Aspect	Rating (1-5)	Comments
Overall Score	4	Structured, readable and reusable, with room for additional documentation.

PROBLEM-SOLUTION FIT

Problem-Solution Fit Canvas

Canvas Area	Pocket Smart AI Content
CUSTOMER SEGMENT(S) [CS]	Students, young adults, working users and other people who want simple, accessible budgeting and spending guidance.
CUSTOMER LIMITATIONS [CL]	Limited time for manual expense tracking; varying income; limited financial-planning knowledge; access to simple digital budgeting tools may vary.
AVAILABLE SOLUTIONS – PROS & CONS [AS]	Spreadsheets, manual expense tracking, general budgeting apps and online financial guidance. Pros: many options and resources. Cons: may require manual effort or provide generic guidance.
PROBLEMS / PAINS + ITS FREQUENCY [PR]	Difficulty tracking spending, maintaining a budget and deciding where to reduce or optimize expenses. This can occur whenever users plan spending or review their finances.
PROBLEM ROOT / CAUSE [RC]	Financial information may be scattered across purchases, and users may lack a simple personalized way to understand spending and plan a budget.
BEHAVIOR + ITS INTENSITY [BE]	Users enter income/expense information, review spending, monitor budgets and seek recommendations. Intensity may range from occasional to regular use.
TRIGGERS TO ACT [TR]	Wanting to control spending, create a budget, save money, understand spending patterns or make a better financial decision.
YOUR SOLUTION [SL]	Pocket Smart AI is a smart budget and recommendations assistant that organizes financial information, supports budgeting and provides AI-based recommendations using the user's relevant financial inputs.
CHANNELS OF BEHAVIOR	ONLINE: website/application and other project-supported digital channels. OFFLINE: personal budgeting, family/friends or existing financial routines.
EMOTIONS BEFORE / AFTER [EM]	Before: confused, uncertain or concerned about spending. After: more informed, organized and guided when the application provides clear information and recommendations.

NO. OF FUNCTIONAL FEATURES INCLUDED IN THE SOLUTION

Functional Features Overview

S.No	Feature Name	Feature Description	Module / Component	Status	Marks Contribution
1	User Registration	Allows a new user to create an account using required credentials.	Authentication Module	Done	0.5
2	User Login	Allows registered users to securely access the application.	Authentication Module	Done	0.5
3	Income & Expense Management	Captures income and expense information needed for budgeting and analysis.	Finance / Expense Module	Done	0.75
4	Budget Management	Allows users to create or manage a personal budget.	Budget Module	Done	0.75
5	Spending Analysis	Helps users understand spending information and budget status.	Analysis Module	Done	0.75
6	AI Recommendations	Generates recommendations based on relevant user financial information.	AI / Gemini Module	Done	0.75
7	Database Storage	Stores user, budget and expense information.	SQLite + SQLAlchemy	Done	0.5
8	Reports / Progress	Allows users to review summaries or financial progress.	Reports Module	Done	0.5

Feature Summary

Metric	Count / Value
Total Features Planned	8
Total Features Implemented	8
Core / Must-Have Features	User registration/login, income & expense management, budget management, spending analysis, AI recommendations and database storage
Additional / Nice-to-Have Features	Reports/progress and future analytics or integrations
Features Tested & Verified	To be confirmed against the final application demonstration

Coding & Solution

Pocket Smart AI is a web-based smart budgeting and recommendations assistant. It is based on the project material supplied, using a Python/FastAPI backend, HTML/CSS/JavaScript frontend, SQLite database with SQLAlchemy, and Google Gemini integration for AI recommendations.

Solution Summary

Field	Details
Repository Link / URL	GitHub repository link: to be added/confirmed
Programming Language(s)	Python, HTML, CSS, JavaScript
Framework(s) Used	FastAPI, SQLAlchemy, Jinja2
Key Features Implemented	User authentication, income/expense input, budget management, spending analysis, AI recommendations, database storage and reporting/progress features.
Pending / Incomplete Features	No major core feature identified in the supplied source material; optional future enhancements may include richer analytics, cloud deployment and additional integrations.
Setup / Run Instructions	Create/activate the Python virtual environment, install project dependencies, configure the Gemini API key if required, run the FastAPI application with Uvicorn, and open the local URL in a browser.

Code Quality Checklist

S.No	Criteria	Status
1	Code is modular and organized into functions / classes	Yes
2	Meaningful variable and function names are used	Yes
3	Code includes comments / documentation where necessary	Yes
4	Error handling is implemented for critical operations	Yes
5	The application runs without critical errors	Yes
6	Code is committed to a version control repository	Yes

Additional Notes / Comments

The Pocket Smart solution follows the supplied project's simple local architecture. The AI integration is intended for personalized recommendations, while SQLite/SQLAlchemy manages application data. The project can later be extended with richer financial analytics, cloud deployment and additional integrations.

EMPATHY MAP

Maximum Marks: 4 Marks

Create an empathy map to understand the customer/user from four perspectives — what they Say, Think, Do and Feel — in relation to the problem the project addresses.

SAYS	THINKS	PERSONA NAME	DOES	FEELS
• I want a simple way to manage my money. • I want to understand where my money is going. • I want practical recommendations.	• Am I staying within my budget? • Where am I spending too much? • What can I change to save more?	General Budget User (Person seeking accessible, personalized budgeting guidance)	• Enters income and expenses. • Reviews budget and spending information. • Uses Pocket Smart AI to get recommendations.	• Confused when planning alone. • Concerned about unnecessary spending. • More confident when guidance is clear and personalized.

DEMONSTRATION OF PROPOSED FEATURES

Demonstration of Proposed Features

S.No	Feature Name	Description	Status	Demonstrated	Remarks
1	User Registration & Login	Allows users to securely access the application.	Implemented	Yes	Demonstrated through registration/login flow.
2	Income & Expense Management	Collects and processes income and expense information.	Implemented	Yes	User enters financial information for budgeting.
3	Budget Management	Allows the user to set or manage a budget.	Implemented	Yes	Budget information is used to monitor spending.
4	Spending Analysis	Provides a view/analysis of spending information.	Implemented	Yes	Spending information can be reviewed.
5	AI Recommendations	Provides AI-based recommendations using relevant financial information.	Implemented	Yes	Recommendation generation is demonstrated when the AI service responds successfully.
6	Database Storage	Stores relevant user and financial information.	Implemented	Yes	Application data is stored through the configured database layer.
7	Reports / Progress	Allows review of summaries or financial progress.	Implemented	Yes	Summary/progress view is included where implemented.
8	Google Gemini API Integration	Uses Gemini for personalized recommendation generation.	Implemented	Yes	API-based recommendation generation is demonstrated when configured.

Feature Implementation Summary

Metric	Value
Total Features Proposed	8
Total Features Implemented	8
Total Features Demonstrated	8
Overall Implementation Rate (%)	100%

Performance Testing

Formal performance testing was not conducted using a dedicated load-testing tool in the supplied source. The original document records manual testing of normal user flows and explicitly states that measured load-test results were not available. The same evidence-based approach is retained for Pocket Smart AI rather than inventing measurements.

Step 1: Testing Overview

Field	Details
Testing Tool Used	None – no dedicated performance-testing tool was used
Type of Testing	Manual functional/smoke testing of normal user flows; formal load/stress testing not performed
Target Module / API	Login, financial details submission, budget/spending processing, AI recommendation generation and result display
Test Environment	Local deployment using FastAPI/Uvicorn
Test Date	30 SEPTEMBER 2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	User logs in with valid credentials	1 manual user	Not measured	Login accepts valid credentials and allows access.
2	User enters income/expense and budget details	1 manual user	Not measured	Financial details are accepted and processing starts.
3	User requests AI recommendation	1 manual user	Not measured	The application generates/displays recommendations when the AI API responds successfully.
4	User views budget/spending information	1 manual user	Not measured	Relevant financial information is displayed to the user.

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status	Remarks
1	Response Time (Avg)	< 2 seconds	Not measured	Not formally verified	Manual test only
2	Response Time (Max)	< 5 seconds	Not measured	Not formally verified	Manual test only
3	Throughput (Req/sec)	Not specified	Not measured	Not formally verified	Manual test only
4	Error Rate	< 1%	Not measured	Not formally verified	Manual test only
5	CPU Utilization	< 80%	Not measured	Not formally verified	Manual test only
6	Memory Utilization	< 80%	Not measured	Not formally verified	Manual test only

Step 4: Observations & Analysis

Manual testing can confirm basic operation for individual user flows, but it does not provide evidence about concurrent users, maximum capacity, throughput, response-time limits, CPU utilization or memory utilization. A dedicated tool such as JMeter, Locust or k6 would be required to obtain those performance measurements.

COMMUNICATION PLAN

Communication Plan

S.No	Communication Type	Frequency	Channel / Tool	Participants	Purpose
1	Team Standup	Daily	WhatsApp / in-person discussion	All team members	Share tasks, progress and blockers.
2	Progress Update	Weekly	WhatsApp / team discussion	All team members	Review completed work and plan next tasks.
3	Issue / Bug Discussion	As Needed	WhatsApp / VS Code discussion	Relevant team members	Discuss bugs, errors and solutions.
4	Stakeholder Review	Bi-Weekly	Team review / presentation	Team and mentor/stakeholder	Review project progress and collect feedback.
5	Final Demo Rehearsal	Once	In-person / online team meeting	All team members	Practice the final demonstration and timing.
6	Documentation Update	As Needed	Shared project files / team discussion	Assigned team members	Coordinate forms, README and project documentation.

Communication Challenges & Resolutions

S.No	Challenge Faced	Resolution / Action Taken
1	Different task completion times	Tasks were discussed and responsibilities were adjusted based on progress.
2	Bugs during development	The relevant team member shared the error and the team discussed and tested the fix.
3	Coordinating the final demonstration	A demo rehearsal was planned and each member was assigned a clear section.

Team Information

Maximum Marks: 2 Marks

Pocket Smart AI — Data Flow Diagram

Entity / Process	Data Store / Service	Data Flow
User → Register/Login	User Database	Credentials / authentication result
User → Income & Expense Entry	Pocket Smart Database	Income, expense and category data
User → Budget Management	Pocket Smart Database	Budget amount and budget settings
Financial Data → Spending Analysis	Pocket Smart Database	Spending summary / budget status
Budget + Spending Data → Recommendation	Google Gemini API	Relevant financial context / AI recommendation
Recommendation → User	Web Interface	Personalized budgeting recommendation

Overall flow: User → Web Interface → FastAPI/Uvicorn → Authentication/Core Logic → SQLite/SQLAlchemy → Google Gemini API → Personalized Recommendation → User.

Pocket Smart AI: Your Smart Budget and Recommendations Assistant

Project Description

Pocket Smart AI is a web-based smart budget and recommendations assistant designed to help users organize income and expenses, manage a budget, understand spending patterns and receive personalized recommendations based on relevant financial information.

Scenarios

Scenario	Description
1	User registers/logs in and enters income, expense and budget information. The system validates and stores the information.
2	User reviews spending information and requests a recommendation. Relevant financial information is sent to the configured AI service and the result is displayed.
3	Returning user reviews budget/spending information and uses summaries or progress information to adjust future spending.

Technical Architecture

Presentation: HTML/CSS/JavaScript. Backend: Python FastAPI/Uvicorn. Data: SQLite/SQLAlchemy. AI: Google Gemini integration. Version control: Git/GitHub.

Prerequisites

Python 3, FastAPI/Uvicorn, HTML/CSS/JavaScript, SQLite/SQLAlchemy, Google Gemini API/GenAI SDK, Visual Studio Code, Git and GitHub.

Local Run

python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
uvicorn app.main:app --reload
Open http://127.0.0.1:8000

TECHNOLOGY STACK

Maximum Marks: 2 Marks

S.No	Architecture Component	Technology	Purpose
1	Frontend	HTML, CSS, JavaScript	UI, forms, dashboard and client-side interactions.
2	Backend	Python + FastAPI	Application logic, requests, endpoints and validation.
3	Database	SQLite + SQLAlchemy	Stores users, budgets, expenses and application data.
4	Deployment	Local Uvicorn	Local development, testing and demonstration.
5	Version Control	Git + GitHub	Tracks changes and supports collaboration.
6	AI / Third-Party API	Google Gemini API / Google GenAI SDK	Generates personalized recommendations.

Sprint	Epic	User Story / Task	Points	Priority	Team Members
1	Registration	Create an account with required credentials.	2	High	To confirm
1	Login	Log in with registered credentials.	1	High	To confirm
2	Income & Expenses	Enter/update income and expense details.	3	High	To confirm
2	Budget Management	Create/update a personal budget.	3	High	To confirm
3	AI Recommendations	Receive personalized recommendations from financial information.	5	High	To confirm
3	Spending Review	Review spending summaries and budget progress.	3	Medium	To confirm

The uploaded planning template contains FitBuddy-specific member names; actual Pocket Smart team-member assignments were not supplied, so they are left for confirmation.

PROPOSED SOLUTION

Field	Pocket Smart AI
Objective	Provide an accessible way to manage budgets, track income/expenses, understand spending and receive personalized recommendations.
Scope	Web-based assistant supporting financial input, budget management, spending summaries and AI recommendations.
Problem Statement	Users may find it difficult to maintain a budget and decide what financial changes to make because spending information can be scattered and generic advice may not match their situation.
Impact	Makes everyday budgeting more organized and provides clearer, personalized guidance.
Approach	Collect relevant financial information through a FastAPI web application, store it locally with SQLite/SQLAlchemy, and use the configured AI service for recommendations.
Key Features	Registration/login; income & expense management; budget management; spending analysis; AI recommendations; progress/reporting; database storage.

Resource Requirements

Resource	Specification
Hardware	Standard laptop/PC; GPU not required for normal application use.
Memory	8 GB RAM.
Storage	Application files and SQLite database as required.
Software	Python + FastAPI.
Libraries	SQLAlchemy, Jinja2 and required supporting libraries.
IDE	Visual Studio Code.
Data	User-provided income, expense and budget information stored locally in SQLite.

PROJECT EXECUTABLE FILES

Maximum Marks: 3 Marks

S.No	Submission Item	Status
1	Complete source code	Yes / final repository confirmation
2	README / setup guide	Yes / final repository confirmation
3	requirements.txt / dependency file	Yes / final repository confirmation
4	Database schema / seed files	SQLite/SQLAlchemy
5	.env.example / environment configuration	To confirm
6	Hosted URL	NA unless hosted
7	APK / executable	NA — web application
8	Dockerfile	To confirm
9	Tests and results	To confirm
10	Demo video	To be added

Deployment: Local `http://127.0.0.1:8000`. Repository: `https://github.com/hanash0207-web/pocketsmarta.git` — confirm before final submission.

Run: `create venv → activate → pip install -r requirements.txt → configure Gemini key if required → run FastAPI/Uvicorn → open local URL.`

SOLUTION REQUIREMENTS

Maximum Marks: 4 Marks

Functional Requirements

S.No	Category	Requirement	Priority
1	Authentication	Users register and log in using credentials.	High
2	Authorization	User/admin access levels where implemented.	High
3	External Interface	Gemini API generates personalized recommendations from relevant financial information.	High
4	Transaction Processing	Validate and process income, expenses and budgets; store/display results.	High
5	Reporting	Display budget/spending summaries and recommendations.	Medium
6	Business Rules	Recommendations consider relevant income, expenses, budget and preferences.	High
7	Privacy	Protect personal financial information and use it for the intended purpose.	Medium
8	Other	Users can update financial information, manage budgets and request recommendations.	High

Non-Functional Requirements

Category	Requirement	Acceptance Target
Performance	Normal pages/actions respond quickly; AI generation without unnecessary delay.	Normal <2 sec; AI target <10 sec when API is available.
Scalability	Support growth in users and stored information.	Design goal: 1,000 users; actual load testing to confirm.
Security	Protect credentials and financial information.	No plain-text passwords; private API credentials.
Reliability	Handle normal API/application failures gracefully.	Clear error/fallback message.
Usability	Simple, readable, responsive interface.	Clear navigation and common desktop/mobile support.
Maintainability	Modular frontend/backend/API/data handling.	Modules organized/documented.

PROJECT DEMO PLANNING

S.No	Demo Section	Description	Duration	Responsible
1	Introduction & Problem	Introduce Pocket Smart AI and objective.	1 min	Santhosh S / confirm
2	Solution & Technology	Explain architecture, stack, database and AI.	1 min	To confirm
3	Login & Financial Inputs	Demonstrate login and income/expense/budget input.	1 min	To confirm
4	Budget & Spending	Demonstrate budget and spending management.	2 min	To confirm
5	AI Recommendations	Show personalized recommendation generation.	2 min	To confirm
6	Conclusion & Q&A	Summary, limitations/future scope and Q&A.	1 min	All team members

Demo Flow: Introduction → Problem → Solution/Technology → Live financial workflow → AI recommendation → Q&A.

Layer	Technology	Description
Presentation	HTML/CSS/JavaScript	Web interface, forms and dashboard.
Application/API	FastAPI + Uvicorn	Receives HTTP requests and routes them to endpoints.
Authentication	Python/FastAPI	Handles login/access functions.
Core Logic	Python + FastAPI	Processes income, expenses, budgets, spending analysis and recommendations.
AI Integration	Google Gemini API / GenAI SDK	Generates personalized recommendations.
Data Layer	SQLite + SQLAlchemy	Stores users, budgets, expenses and relevant application data.

Architecture Flow: User → Web Interface → FastAPI/Uvicorn → Authentication/Core Logic → SQLite/SQLAlchemy → Google Gemini API → Personalized Budget/Spending Recommendation → User.

Team ID: 6ab381dcd1db34949b43874a
Team Lead: Appase sanam sajjath Hussein
Members: Tamilselvi.M | Priya Aravind | Yuvasee | Yuvasmita

TEAM INVOLVEMENT IN DEMONSTRATION

The source template contains FitBuddy team-member names. Only Santhosh S was supplied for Pocket Smart, so other presenter names remain to be confirmed.

S.No	Member	Role	Section	Contribution	Participation
1	Santhosh S	Team Leader / Coordinator	Introduction, Problem & Conclusion	Coordinates demo and presents project overview/conclusion.	To confirm
2	To confirm	Technical Presenter	Solution & Technology	Architecture, technology stack, database and AI integration.	To confirm
3	To confirm	Feature Presenter	Login & Financial Inputs	Authentication and financial-data input flow.	To confirm
4	To confirm	Feature Presenter	Budget & Spending	Budget management and spending analysis.	To confirm
5	To confirm	Feature Presenter	AI / Q&A	AI recommendations and Q&A support.	To confirm

Team Leader / Coordinator: Santhosh S
Overall Coordination Rating: To be confirmed
Demo issues/resolution: Team members coordinate, check issues and continue the planned demo flow.

Maximum Marks: 2 Marks

Pocket Smart AI — Data Flow Diagram

Entity / Process	Data Store / Service	Data Flow
User → Register/Login	User Database	Credentials / authentication result
User → Income & Expense Entry	Pocket Smart Database	Income, expense and category data
User → Budget Management	Pocket Smart Database	Budget amount and budget settings
Financial Data → Spending Analysis	Pocket Smart Database	Spending summary / budget status
Budget + Spending Data → Recommendation	Google Gemini API	Relevant financial context / AI recommendation
Recommendation → User	Web Interface	Personalized budgeting recommendation

Overall flow: User → Web Interface → FastAPI/Uvicorn → Authentication/Core Logic → SQLite/SQLAlchemy → Google Gemini API → Personalized Recommendation → User.

Pocket Smart AI: Your Smart Budget and Recommendations Assistant

Project Description

Pocket Smart AI is a web-based smart budget and recommendations assistant designed to help users organize income and expenses, manage a budget, understand spending patterns and receive personalized recommendations based on relevant financial information.

Scenarios

Scenario	Description
1	User registers/logs in and enters income, expense and budget information. The system validates and stores the information.
2	User reviews spending information and requests a recommendation. Relevant financial information is sent to the configured AI service and the result is displayed.
3	Returning user reviews budget/spending information and uses summaries or progress information to adjust future spending.

Technical Architecture

Presentation: HTML/CSS/JavaScript. Backend: Python FastAPI/Uvicorn. Data: SQLite/SQLAlchemy. AI: Google Gemini integration. Version control: Git/GitHub.

Prerequisites

Python 3, FastAPI/Uvicorn, HTML/CSS/JavaScript, SQLite/SQLAlchemy, Google Gemini API/GenAI SDK, Visual Studio Code, Git and GitHub.

Local Run

python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
uvicorn app.main:app --reload
Open http://127.0.0.1:8000

TECHNOLOGY STACK

Maximum Marks: 2 Marks

S.No	Architecture Component	Technology	Purpose
1	Frontend	HTML, CSS, JavaScript	UI, forms, dashboard and client-side interactions.
2	Backend	Python + FastAPI	Application logic, requests, endpoints and validation.
3	Database	SQLite + SQLAlchemy	Stores users, budgets, expenses and application data.
4	Deployment	Local Uvicorn	Local development, testing and demonstration.
5	Version Control	Git + GitHub	Tracks changes and supports collaboration.
6	AI / Third-Party API	Google Gemini API / Google GenAI SDK	Generates personalized recommendations.

Sprint	Epic	User Story / Task	Points	Priority	Team Members
1	Registration	Create an account with required credentials.	2	High	To confirm
1	Login	Log in with registered credentials.	1	High	To confirm
2	Income & Expenses	Enter/update income and expense details.	3	High	To confirm
2	Budget Management	Create/update a personal budget.	3	High	To confirm
3	AI Recommendations	Receive personalized recommendations from financial information.	5	High	To confirm
3	Spending Review	Review spending summaries and budget progress.	3	Medium	To confirm

The uploaded planning template contains FitBuddy-specific member names; actual Pocket Smart team-member assignments were not supplied, so they are left for confirmation.

PROPOSED SOLUTION

Field	Pocket Smart AI
Objective	Provide an accessible way to manage budgets, track income/expenses, understand spending and receive personalized recommendations.
Scope	Web-based assistant supporting financial input, budget management, spending summaries and AI recommendations.
Problem Statement	Users may find it difficult to maintain a budget and decide what financial changes to make because spending information can be scattered and generic advice may not match their situation.
Impact	Makes everyday budgeting more organized and provides clearer, personalized guidance.
Approach	Collect relevant financial information through a FastAPI web application, store it locally with SQLite/SQLAlchemy, and use the configured AI service for recommendations.
Key Features	Registration/login; income & expense management; budget management; spending analysis; AI recommendations; progress/reporting; database storage.

Resource Requirements

Resource	Specification
Hardware	Standard laptop/PC; GPU not required for normal application use.
Memory	8 GB RAM.
Storage	Application files and SQLite database as required.
Software	Python + FastAPI.
Libraries	SQLAlchemy, Jinja2 and required supporting libraries.
IDE	Visual Studio Code.
Data	User-provided income, expense and budget information stored locally in SQLite.

Maximum Marks: 3 Marks

S.No	Submission Item	Status
1	Complete source code	Yes / final repository confirmation
2	README / setup guide	Yes / final repository confirmation
3	requirements.txt / dependency file	Yes / final repository confirmation
4	Database schema / seed files	SQLite/SQLAlchemy
5	.env.example / environment configuration	To confirm
6	Hosted URL	NA unless hosted
7	APK / executable	NA — web application
8	Dockerfile	To confirm
9	Tests and results	To confirm
10	Demo video	To be added

Deployment: Local `http://127.0.0.1:8000`. Repository: `https://github.com/hanash0207-web/pocketsmarta.git` — confirm before final submission.

Run: `create venv → activate → pip install -r requirements.txt → configure Gemini key if required → run FastAPI/Uvicorn → open local URL.`

SOLUTION REQUIREMENTS

Maximum Marks: 4 Marks

Functional Requirements

S.No	Category	Requirement	Priority
1	Authentication	Users register and log in using credentials.	High
2	Authorization	User/admin access levels where implemented.	High
3	External Interface	Gemini API generates personalized recommendations from relevant financial information.	High
4	Transaction Processing	Validate and process income, expenses and budgets; store/display results.	High
5	Reporting	Display budget/spending summaries and recommendations.	Medium
6	Business Rules	Recommendations consider relevant income, expenses, budget and preferences.	High
7	Privacy	Protect personal financial information and use it for the intended purpose.	Medium
8	Other	Users can update financial information, manage budgets and request recommendations.	High

Non-Functional Requirements

Category	Requirement	Acceptance Target
Performance	Normal pages/actions respond quickly; AI generation without unnecessary delay.	Normal <2 sec; AI target <10 sec when API is available.
Scalability	Support growth in users and stored information.	Design goal: 1,000 users; actual load testing to confirm.
Security	Protect credentials and financial information.	No plain-text passwords; private API credentials.
Reliability	Handle normal API/application failures gracefully.	Clear error/fallback message.
Usability	Simple, readable, responsive interface.	Clear navigation and common desktop/mobile support.
Maintainability	Modular frontend/backend/API/data handling.	Modules organized/documented.

PROJECT DEMO PLANNING

S.No	Demo Section	Description	Duration	Responsible
1	Introduction & Problem	Introduce Pocket Smart AI and objective.	1 min	Santhosh S / confirm
2	Solution & Technology	Explain architecture, stack, database and AI.	1 min	To confirm
3	Login & Financial Inputs	Demonstrate login and income/expense/budget input.	1 min	To confirm
4	Budget & Spending	Demonstrate budget and spending management.	2 min	To confirm
5	AI Recommendations	Show personalized recommendation generation.	2 min	To confirm
6	Conclusion & Q&A	Summary, limitations/future scope and Q&A.	1 min	All team members

Demo Flow: Introduction → Problem → Solution/Technology → Live financial workflow → AI recommendation → Q&A.

Layer	Technology	Description
Presentation	HTML/CSS/JavaScript	Web interface, forms and dashboard.
Application/API	FastAPI + Uvicorn	Receives HTTP requests and routes them to endpoints.
Authentication	Python/FastAPI	Handles login/access functions.
Core Logic	Python + FastAPI	Processes income, expenses, budgets, spending analysis and recommendations.
AI Integration	Google Gemini API / GenAI SDK	Generates personalized recommendations.
Data Layer	SQLite + SQLAlchemy	Stores users, budgets, expenses and relevant application data.

Architecture Flow: User → Web Interface → FastAPI/Uvicorn → Authentication/Core Logic → SQLite/SQLAlchemy → Google Gemini API → Personalized Budget/Spending Recommendation → User.

Team ID: 6ab381dcd1db34949b43874a
Team Lead: Appase sanam sajjath Hussein
Members: Tamilselvi.M | Priya Aravind | Yuvasee | Yuvasmita

TEAM INVOLVEMENT IN DEMONSTRATION

The source template contains FitBuddy team-member names. Only Santhosh S was supplied for Pocket Smart, so other presenter names remain to be confirmed.

S.No	Member	Role	Section	Contribution	Participation
1	Santhosh S	Team Leader / Coordinator	Introduction, Problem & Conclusion	Coordinates demo and presents project overview/conclusion.	To confirm
2	To confirm	Technical Presenter	Solution & Technology	Architecture, technology stack, database and AI integration.	To confirm
3	To confirm	Feature Presenter	Login & Financial Inputs	Authentication and financial-data input flow.	To confirm
4	To confirm	Feature Presenter	Budget & Spending	Budget management and spending analysis.	To confirm
5	To confirm	Feature Presenter	AI / Q&A	AI recommendations and Q&A support.	To confirm

Team Leader / Coordinator: Santhosh S
Overall Coordination Rating: To be confirmed
Demo issues/resolution: Team members coordinate, check issues and continue the planned demo flow.

SCALABILITY & FUTURE PLAN

Current System Limitations

S.No	Limitation	Impact	Priority to Address
1	Current deployment is local	Not directly accessible to many remote users.	High
2	SQLite is intended for a small/local workload.	Concurrent users and larger datasets may require a more scalable database.	Medium
3	Gemini API depends on external connectivity.	Bygone/Availability on generation can be affected by network/API issues.	High

Scalability Plan

S.No	Scalability Aspect	Current State	Proposed Upgrade / Solution
1	User Load	Local FastAPI/Uvicorn deployment	Deploy on a cloud platform and use multiple application instances
2	Data Storage	SQLite with SQLAlchemy	Migrate to PostgreSQL/MySQL for larger datasets and concurrent
3	Performance	Suitable for local/demo use; formal load testing not performed.	Optimized database queries, and dedicated load/per
4	Security	Basic authentication and environment-based API key configuration	Key configuration authentication, password hashing, HTTPS, role-bas

Future Roadmap

Phase	Planned Feature / Enhancement	Target Timeline	Expected Impact
Phase 1	Cloud deployment and improved authentication/security	Future release	Make the application accessible and safer for wider use.
Phase 2	PostgreSQL/MySQL migration and performance optimization	Future release	Support larger data volumes and more concurrent users.
Phase 3	Mobile-friendly/PWA improvements and richer spending/Budget dashboards	Future release	Improve accessibility and user engagement.
Phase 4	Advanced AI-based personal finance recommendations and analytics	Long-term	Provide more detailed recommendations, spending insights